

History of Major Imperative Languages

Family tree of imperative languages

- FORTRAN family
- ALGOL family
- COBOL family

TPK Algorithm

The TPK algorithm, authored by Luis Trabb Pardo and Donald E. Knuth, illustrates several important features of programming languages, including:

- Variable declarations
- Data types
- Arrays
- Mathematical calculations
- Control constructs: Loops, conditional branches, compound statements, gotos
- Function definitions, function calls, and return types
- Input/output
- Comments

We'll see the TPK algorithm implemented in a number of different languages.

Algorithm

Read 11 numbers into an array (or similar data structure)

In reverse order, for each number read:

 Compute $y = \sqrt{|x|} + 5x^3$

 If $y > 400$ then

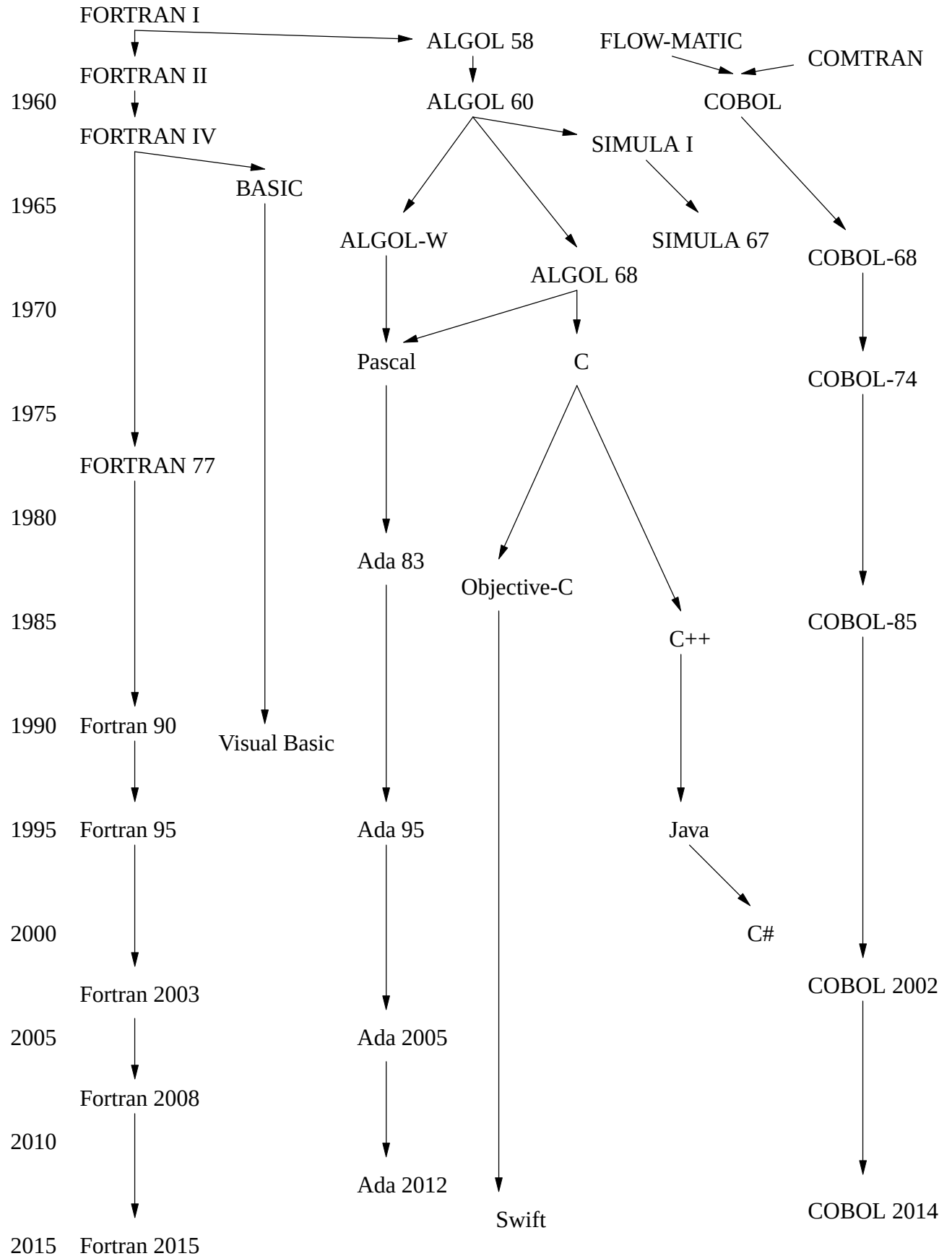
 print "TOO LARGE"

 Else

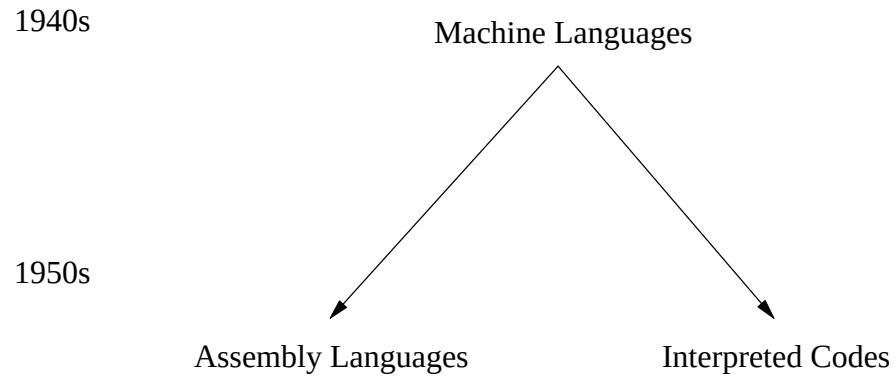
 print x, y

Sources for sample programs

- The family tree diagram of early languages is largely based on Robert W. Sebesta, *Concepts of Programming Languages*, Benjamin/Cummings, Redwood City, CA, 1st edition, 1989, and Robert W. Sebesta, *Concepts of Programming Languages*, Benjamin/Cummings, Redwood City, CA, 12th edition, 2019.
- The TPK algorithm and the Short Code, FLOW-MATIC, FORTRAN, and ALGOL 60 sample programs are from Donald E. Knuth & Luis Trabb Pardo, “The Early Development of Programming Languages”, in *A History of Computing in the Twentieth Century*, ed. N. Metropolis, J. Howlett, and Gian-Carlo Rota, Academic Press, 1980. The algorithm is, in the authors’ words, “quite useless”, but it illustrates useful features of languages for comparison.
- The COBOL implementation of the TPK algorithm was written by Steven Gavarone, a Rowan University student, in 2010.
- The other COBOL example is from Robert W. Sebesta, *Concepts of Programming Languages*, Benjamin/Cummings, Redwood City, CA, 2nd edition, 1993.
- The Pascal, Ada, BASIC, and C implementations of the TPK algorithm were written by Nancy Tinkham.



Before the high-level languages...



A UNIVAC Short Code Example (1952)

00	i = 10		00	00	00	W0	03	Z2
01	0: y = ($\sqrt{-}$ abs t) + 5 cube t		T0	02	07	Z5	11	T0
02			00	Y0	03	09	20	06
03	y 400 if $\leq$ to 1		00	00	00	Y0	Z3	41
04	i print, 'TOO LARGE' print-and-return		00	00	Z4	59	W0	58
05	0 0 if = to 2		00	00	00	Z0	Z0	72
06	1: i print, y print-and-return		00	00	Y0	59	W0	58
07	2: T0 U0 shift		00	00	00	T0	U0	99
08	i = i - 1		00	W0	03	W0	01	Z1
09	0 i if $\leq$ to 0		00	00	00	Z0	W0	40
10	stop		00	00	00	00	ZZ	08

Tables used by program (partial list)

Variable	Symbol	Constant	Symbol	Operation	Symbol
<i>i</i>	W0	0.0	Z0	–	01
<i>t</i>	T0	1.0	Z1	)	02
<i>y</i>	Y0	10.0	Z2	=	03
		400.0	Z3	abs	06
		TOO_LARGE	Z4	+	07
		5.0	Z5	(	09
				if $\leq$ to <i>n</i>	4 <i>n</i>

Inputs in U0, T9, T8, ..., T0

High-Level Languages

- Portable
- Readable by humans

A "FORTRAN 0" Example (1954)

```
DIMENSION A(11)
READ A
2   DO 3, 8, 11 J = 1,11
3   I = 11 - J
    Y = SQRT(ABS(A(I+1))) + 5 * A(I + 1)**3
    IF (400 >= Y) 8, 4
4   PRINT I, 999
    GO TO 2
8   PRINT I, Y
11  STOP
```

A FORTRAN I Example (1957)

```
C   TPK - LUIS TRABB PARDO AND DONALD E. KNUTH
      FUNF(T) = SQRTF(ABSF(T)) + 5.0*T**3
      DIMENSION A(11)
1     FORMAT(6F12.4)
      READ 1, A
      DO 10 J = 1,11
        I = 11 - J
        Y = FUNF(A(I+1))
        IF (400.0 - Y) 4, 8, 8
4       PRINT 5, I
5       FORMAT(I10, 10H TOO LARGE)
        GO TO 10
8       PRINT 9, I, Y
9       FORMAT(I10, F12.7)
10      CONTINUE
11     STOP 52525
```

A BASIC Example (IBM PC BASIC 1982)

```
10  REM Trabb Pardo and Knuth's TPK Algorithm
20  DEF FNF(T) = SQRT(ABS(T)) + 5.0 * T ^ 3
30  DIM A(11)
40  FOR I = 0 TO 10
50      INPUT A(I)
60  NEXT I
70  FOR I = 10 TO 0 STEP -1
80      Y = FNF(A(I))
90      IF Y > 400 THEN
100         PRINT I, "TOO LARGE"
110     ELSE
120         PRINT I, Y
130 NEXT I
140 END
```

An ALGOL 60 Example (1960)

```
TPK: begin integer i; real y; real array a[0:10];  
      real procedure f(t); real t; value t;  
          f := sqrt(abs(t)) + 5 × t ↑ 3;  
      for i := 0 step 1 until 10 do  
          read(a[i]);  
      for i := 10 step -1 until 0 do  
          begin  
              y := f(a[i]);  
              if y > 400 then  
                  write(i, "TOO LARGE")  
              else  
                  write(i, y);  
          end  
      end.
```

A Pascal Example (1971)

```
program TPK;  
{ TPK algorithm - Luis Trabb Pardo & Donald E. Knuth }  
  
  var  
    i: integer;  
    y: real;  
    a: array[0..10] of real;  
  
  function power(base, exponent: real): real;  
    begin  
      power := exp(exponent * ln(base))  
    end;  
  
  function f(t: real): real;  
    begin  
      f := sqrt(abs(t)) + 5 * power(t, 3)  
    end;  
  
  begin  
    for i := 0 to 10 do  
      readln(a[i]);  
    for i := 10 downto 0 do  
      begin  
        y := f(a[i]);  
        if y > 400 then  
          writeln(i, 'TOO LARGE')  
        else  
          writeln(i, ' ', y)  
        end  
      end  
  
  end.
```

An Ada Example (1995)

```
with Ada.Text_IO, Ada.Integer_Text_IO, Ada.Float_Text_IO;
use Ada.Text_IO, Ada.Integer_Text_IO, Ada.Float_Text_IO;
with Ada.Numerics.Elementary_Functions;
use Ada.Numerics.Elementary_Functions;
```

procedure TPK is

-- TPK algorithm - Luis Trabb Pardo & Donald E. Knuth

 y: float;

 a: array(0..10) of float;

 function f(t: in float) return float is

 begin

 return Sqrt(abs t) + 5.0 * t ** 3;

 end f;

 begin

 for i in 0..10 loop

 Get(a(i));

 end loop;

 for i in reverse 0..10 loop

 y := f(a(i));

 if y > 400.0 then

 Put(i, Width => 1);

 Put_Line(" too large");

 else

 Put(i, Width => 1);

 Put(" ");

 Put(y, Fore => 1, Aft => 2, Exp => 0);

 New_Line;

 end if;

 end loop;

 end TPK;

A C Example (1972)

/* TPK algorithm - Luis Trabb Pardo & Donald E. Knuth

```
#include <stdio.h>
```

```
#include <math.h>
```

```
double f(double t) {  
    return sqrt(fabs(t)) + 5.0 * pow(t, 3);  
}
```

```
int main() {  
    double y;  
    double a[11];  
    int i;  
  
    for (i = 0; i <= 10; i++)  
        scanf("%lf", &a[i]);  
  
    for (i = 10; i >= 0; i--) {  
        y = f(a[i]);  
        if (y > 400.0)  
            printf("%d too large\n", i);  
        else  
            printf("%d %lf\n", i, y);  
    }  
  
    return 0;  
}
```

Sample FLOW-MATIC operations

COMPARE PART-NUMBER (A) TO PART-NUMBER (B); IF
GREATER GO TO OPERATION 13; IF EQUAL GO TO OPERATION
4; OTHERWISE GO TO OPERATION 2 .

READ-ITEM B; IF END OF DATA GO TO OPERATION 10.

COBOL example (1960)

IDENTIFICATION DIVISION.

PROGRAM-ID. PRODUCE-REORDER-LISTING.

ENVIRONMENT DIVISION.

CONFIGURATION SECTION.

SOURCE-COMPUTER. DEC-VAX.

OBJECT-COMPUTER. DEC-VAX.

INPUT-OUTPUT SECTION.

FILE-CONTROL.

 SELECT BAL-FWD-FILE

 ASSIGN TO READER.

 SELECT REORDER-LISTING

 ASSIGN TO LOCAL-PRINTER.

DATA DIVISION.

FILE SECTION.

FD BAL-FWD-FILE
LABEL RECORDS ARE STANDARD
RECORD CONTAINS 80 CHARACTERS.

01 BAL-FWD-CARD.
02 BF-ITEM-NO PICTURE IS 9(5).
02 BF-ITEM-DESC PICTURE IS X(20).
02 FILLER PICTURE IS X(5).
02 BF-UNIT-PRICE PICTURE IS 999V99.
02 BF-REORDER-POINT PICTURE IS 9(5).
02 BF-ON-HAND PICTURE IS 9(5).
02 BF-ON-ORDER PICTURE IS 9(5).
02 FILLER PICTURE IS X(30).

FD REORDER-LISTING
LABEL RECORDS ARE STANDARD
RECORD CONTAINS 132 CHARACTERS.

01 REORDER-LINE.
02 RL-ITEM-NO PICTURE IS Z(5).
02 FILLER PICTURE IS X(5).
02 RL-ITEM-DESC PICTURE IS X(20).
02 FILLER PICTURE IS X(5).
02 RL-UNIT-PRICE PICTURE IS ZZZ.99.
02 FILLER PICTURE IS X(5).
02 RL-AVAILABLE-STOCK PICTURE IS Z(5).
02 FILLER PICTURE IS X(5).
02 RL-REORDER-POINT PICTURE IS Z(5).
02 FILLER PICTURE IS X(71).

WORKING-STORAGE SECTION.

01 SWITCHES.
02 CARD-EOF-SWITCH PICTURE IS X.
01 WORK-FIELDS.
02 AVAILABLE-STOCK PICTURE IS 9(5).

PROCEDURE DIVISION.

000-PRODUCE-REORDER-LISTING.

OPEN INPUT BAL-FWD-FILE.
OPEN OUTPUT REORDER-LISTING.
MOVE "N" TO CARD-EOF-SWITCH.
PERFORM 100-PRODUCE-REORDER-LINE
 UNTIL CARD-EOF-SWITCH IS EQUAL TO "Y".
CLOSE BAL-FWD-FILE.
CLOSE REORDER-LISTING.
STOP RUN.

100-PRODUCE-REORDER-LINE.

PERFORM 110-READ-INVENTORY-RECORD.
IF CARD-EOF-SWITCH IS NOT EQUAL TO "Y"
 PERFORM 120-CALCULATE-AVAILABLE STOCK
 IF AVAILABLE-STOCK IS LESS THAN BF-REORDER-POINT
 PERFORM 130-PRINT-REORDER-LINE.

110-READ-INVENTORY-RECORD.

READ BAL-FWD-FILE RECORD
AT END
 MOVE "Y" TO CARD-EOF-SWITCH.

120-CALCULATE-AVAILABLE-STOCK.

ADD BF-ON-HAND BF-ON-ORDER
GIVING AVAILABLE-STOCK.

130-PRINT-REORDER-LINE.

MOVE SPACE	TO REORDER-LINE.
MOVE BF-ITEM-NO	TO RL-ITEM-NO.
MOVE BF-ITEM-DESC	TO RL-ITEM-DESC.
MOVE BF-UNIT-PRICE	TO RL-UNIT-PRICE.
MOVE AVAILABLE-STOCK	TO RL-AVAILABLE-STOCK.
MOVE BF-REORDER-POINT	TO RL-REORDER-POINT.
WRITE REORDER-LINE.	

COBOL (implementation of TPK algorithm)

IDENTIFICATION DIVISION.

PROGRAM-ID. TPK.

AUTHOR. Steven Gavarone.

DATE-WRITTEN. 12-Sept-2010.

DATE-COMPILED.

SECURITY. Unlimited access to general public.

```
*****
* tpk.cbl -- Knuth's TPK program in the programming language COBOL
*
* which is  $\text{SQRT}(|x|) + (5 * x^3)$ 
*****
```

ENVIRONMENT DIVISION.

CONFIGURATION SECTION.

DATA DIVISION.

WORKING-STORAGE SECTION.

01	TPK-INPUT	OCCURS 11 TIMES,	PIC S9(3).
01	OPTION-NUMBER		PIC S9(3).
01	ARR-POS		PIC 9(2) VALUE 0.
01	ABS-VALUE		PIC 9(3), USAGE COMP-5.
01	COMP-RESULT		PIC S9(9)V999999 VALUE 0, USAGE COMP-5.
01	DISP-RESULT		PIC +Z(5).9(5).
01	PAUSE-INPUT		PIC X(1) VALUE SPACE.

PROCEDURE DIVISION.

MAIN-PROGRAM SECTION.

0100-MAIN-PROGRAM.

- * Perform loop to gather the input 11 times:
 PERFORM 1000-FILL-ARRAY VARYING ARR-POS FROM 1 BY 1 UNTIL ARR-POS > 11.
 DISPLAY " ".
- * In reverse order, compute the value on each element of the array and output
 PERFORM 2000-COMPUTE-VALUES VARYING ARR-POS FROM 11 BY -1 UNTIL ARR-POS = 0.
 DISPLAY "Press <ENTER> to continue:" WITH NO ADVANCING.
 ACCEPT PAUSE-INPUT.
 STOP RUN.

1000-FILL-ARRAY.

- * Read in the values of the array
 MOVE 0 TO OPTION-NUMBER.
 ACCEPT OPTION-NUMBER.
 MOVE OPTION-NUMBER TO TPK-INPUT(ARR-POS).

2000-COMPUTE-VALUES.

- * Easy Absolute value determination
 MOVE TPK-INPUT(ARR-POS) TO ABS-VALUE.
 IF TPK-INPUT(ARR-POS) < 0
 COMPUTE ABS-VALUE = TPK-INPUT(ARR-POS) * -1.
- * Now the computation
 COMPUTE COMP-RESULT = (ABS-VALUE ** .5) + (5 * (TPK-INPUT(ARR-POS) ** 3))
- * And the output
 IF COMP-RESULT > 400 THEN
 DISPLAY ARR-POS "." SPACE TPK-INPUT(ARR-POS) SPACE "TOO LARGE"
 ELSE
 MOVE COMP-RESULT TO DISP-RESULT
 DISPLAY ARR-POS "." SPACE DISP-RESULT.